arXiv:2405.10637v2 [cs.CL] 4 Jun 2024

Layer-Condensed KV Cache for Efficient Inference of Large Language Models

Haoyi Wu and **Kewei Tu***
School of Information Science and Technology, ShanghaiTech University
Shanghai Engineering Research Center of Intelligent Vision and Imaging
{wuhy1, tukw}@shanghaitech.edu.cn

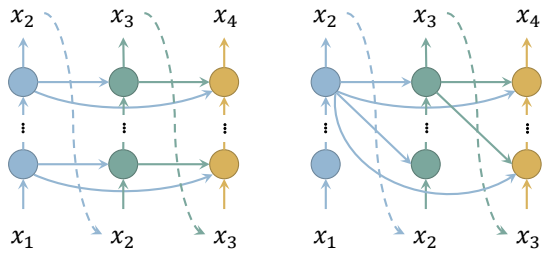
Abstract

Huge memory consumption has been a major bottleneck for deploying high-throughput large language models in real-world applications. In addition to the large number of parameters, the key-value (KV) cache for the attention mechanism in the transformer architecture consumes a significant amount of memory, especially when the number of layers is large for deep language models. In this paper, we propose a novel method that only computes and caches the KVs of a small number of layers, thus significantly saving memory consumption and improving inference throughput. Our experiments on large language models show that our method achieves up to 26× higher throughput than standard transformers and competitive performance in language modeling and downstream tasks. In addition, our method is orthogonal to existing transformer memory-saving techniques, so it is straightforward to integrate them with our model, achieving further improvement in inference efficiency. Our code is available at https://github.com/whyNLP/LCKV.

1 Introduction

High throughput and low latency are essential for deploying large language models (LLMs) in real-world applications (Tillet et al., 2019; Kwon et al., 2023). However, the huge memory consumption of LLMs has been a major bottleneck, preventing a large batch size and high throughput generation. Among the memory-consuming components, the key-value (KV) cache is one of the most significant parts (Pope et al., 2023; Zhang et al., 2023) that takes over 30% of the GPU memory during deployment (Kwon et al., 2023). The KV cache is used to store the keys and values in each transformer layer during generation to avoid re-computation. Its memory consumption is proportional to both the sequence length and the number of layers.

* Corresponding author.



(a) Standard transformer (b) Our model

Figure 1: Illustration of a standard transformer decoder and our model. Each node represents one layer of transformer computation of one token. Each horizontal edge $a \to b$ denotes that the queries at b are paired with the KVs at a.

There have been substantial works on reducing the memory consumption of the KV cache in LLMs. Most of them focus on compressing the KV cache by reducing the length of the cached KV sequence. For example, Jiang et al. (2023a); Li et al. (2023); Mu et al. (2023) compress the prompts to save the memory consumption. Ren et al. (2023) incrementally compress a specified span of tokens into compact ones to reduce the KV cache length. Xiao et al. (2024); Han et al. (2023) propose to store only the KVs of initial and recent tokens in the KV cache. Zhang et al. (2023) propose a dynamic KV cache eviction policy to only keep a small portion of the KV cache in memory.

In this paper, we propose to reduce the memory consumption of the KV cache from a novel perspective that is orthogonal to previous efforts: reducing the number of layers. Specifically, we present a new variant of transformer decoders in which queries of all layers are paired with keys and values of just the top layer, as illustrated in Figure 1. In this way, we only need to cache the keys and values of one layer (vs. tens of layers in a typical LLM), significantly saving memory consumption while introducing no computation overheads dur-

ing inference. In fact, since we only need the keys and values of the top layer, we can omit the key-value computation and discard the key-value parameters for all the other layers, further improving the throughput and reducing the memory consumption as well as the mode size.

We draw our inspiration from the interpretation of the stacking layer structure of a transformer as an iterative process of improving token representation (Wu and Tu, 2023). In this interpretation, the representation at the top layer is the most informative, so it makes sense to attend only to the top layer. We also note the similarity of our idea to the cross-attention mechanism in a standard transformer encoder-decoder, in which all the decoder layers attend to the top encoder layer. However, applying this idea to a decoder has never been attempted before as far as we know.

Although our model presented above achieves very efficient inference, its performance in language modeling and downstream tasks degrades in comparison with standard transformers. Therefore, we further propose to retain standard attention for a small number of layers in our model, which slightly diminishes our saving of the KV cache memory consumption but leads to almost no performance degradation.

Another challenge faced by our model is training. When training a standard transformer decoder, the computation of all the tokens can be fully parallelized. In our model, however, the computation at each token depends on the top layer of the previous token, creating sequential dependencies that spoil parallel training. We address the challenge by deriving an approximate training method that supports parallel training.

Our experiments on Llama (Touvron et al., 2023) show that our model achieves up to 32× larger batch sizes and up to 26× higher throughput than standard transformers for LLMs of 1B–30B parameters; at the same time, our model has competitive performance to standard transformers in language modeling and downstream tasks. We further empirically demonstrate that it is straightforward to integrate our model with other memory-saving techniques like StreamingLLM (Xiao et al., 2024), achieving further improvements in inference efficiency.

We summarize our contributions as follows: 1) we propose a new variant of transformer decoders that reduces the memory consumption of the KV cache by dramatically reducing the number of cached layers; 2) we make parallel training of our model feasible by designing a novel approximate training method; 3) we conduct extensive experiments to verify and analyze the effectiveness and efficiency of our method.

2 Layer-Condensed KV Cache

2.1 Model

As shown in Figure 1(b), we pair the queries of all layers with KVs of only the top layer, so that we do not have to cache or even compute KVs for layers other than the top layer, saving both memory consumption and computation. Furthermore, since we no longer need to compute KVs for these layers, nor do we need to keep the weights W_K, W_V that map hidden representations to KVs for these layers, thus also saving model parameters.

One problem with this method is that, since each token also attends to itself, we need its top-layer KVs for its attention computation at lower layers, but the top-layer cannot be computed until we finish the computation of all the lower layers. A straightforward solution to this cyclic dependency problem is to drop the attention of each token to itself, which is equivalent to masking the diagonal of the attention matrix. Now the first token of the sequence has nothing to attend to, so we just use zero vectors as dummy KVs in its attention computation. Note that even without self-attention of each token, its information can still be incorporated in its bottom-up computation thanks to residual connections. Empirically, we find that the diagonal mask of the attention matrix does not affect the performance of the model.

It has been previously observed that transformers tend to attend to syntactic information in lower layers and semantic information in higher layers (Clark et al., 2019b). Intuitively, applying KVs of the same layer to queries of all layers might break this pattern. Empirically, we do find that our method as described above underperforms standard transformers in language modeling and downstream tasks. A simple yet effective solution to this problem is to retain standard attention for a small number of layers, which we call *warmup layers*, and only apply our method to the rest of the layers. Inspired by the sandwich module of Reid et al. (2021), we propose to keep the top $w/2$ layers and the bottom $w/2$ layers as warmup layers. We empirically find that such a sandwich configuration outperforms alternative configurations and leads to

almost no performance degradation compared with standard transformers.

2.2 Training

Though the inference process of our model is straightforward and almost identical to that of a standard transformer, i.e., decoding one token at a time from left to right, the training process of our model is more complicated. Since each token relies on the KVs of the top layer of previous tokens, it is impossible to train on a sequence of tokens in parallel as in a standard transformer. Below we derive a parallel training process of our model in three steps. For simplicity, we assume no warmup layers (i.e., $w = 0$). It is straightforward to add warmup layers in the derived training process.

2.2.1 From Sequential to Parallel Training

During training, we minimize the cross entropy loss for each token. In our model, the computation of each token depends on the top-layer KVs of its previous tokens. Therefore, for a token sequence of length n, training is done sequentially on a computation graph consisting of n passes of bottom-up transformer computation, as shown in Figure 2(a).

In the following, we present a different training computation graph and show its equivalence to the original computation graph. Specifically, we perform n iterations of bottom-up transformer computation on all tokens in parallel, and in each iteration, we pair the queries of all layers with KVs of the top layer from the previous iteration, thus breaking the sequential dependencies within the iteration. We compute the cross entropy loss only after the last iteration. Note that the first iteration has no previous iteration, and we pair its queries with dummy KVs which are zero vectors. Figure 2(b) shows the new computation graph.

Theorem 1. *The two computation graphs are equivalent in terms of model training.*

We leave the proof of the theorem to Appendix A. Here, we provide an intuitive explanation. We say the computation of a token (w.r.t. its hidden representations and top-layer KVs) is correct in an iteration of the second computation graph if and only if it is identical to that in the first computation graph. Since we have masked the diagonal of the attention matrix, the computation of the first token does not rely on any key or value (except for dummy zero vectors) and thus is correct in every iteration. For the second token, its computation relies on the KVs of the first token and thus is correct

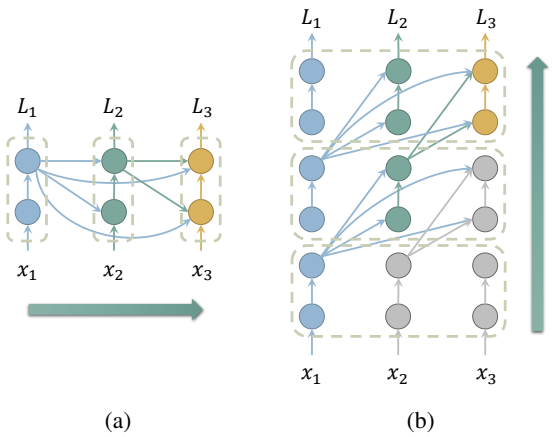


Figure 2: Two equivalent computation graphs for training our model with two layers. (a) Sequential over n tokens. (b) Parallel over n tokens with n iterations. Sub-graphs with the same color (except grey) represent identical data flow. Sub-graphs in grey are unused in loss computation.

starting from the second iteration. In general, the computation of the i-th token relies on the KVs of the first $i-1$ tokens and by induction, it is correct starting from the i-th iteration. Therefore, after n iterations, all the tokens are correctly computed. As a result, the computation sub-graphs of the cross-entropy losses of all the tokens are identical in the two graphs, and hence training processes following the two graphs are equivalent.

Essentially, the second computation graph replaces horizontal dependencies in the first graph with vertical dependencies, and it does not change the length of the longest dependency chain. Consequently, although the second computation graph supports parallel training over all the tokens, it still requires the same n iterations as the first graph. Next, we will trim the iterations first in terms of backpropagation and then in terms of forward propagation.

2.2.2 Backpropagation: Gradient Stopping

We compute the cross entropy loss after the last iteration, which backpropagates through n iterations, resulting in a large computation graph that is impossible to fit into GPU memory for large n. To solve the issue, we follow the practice of gradient stopping in Transformer-XL (Dai et al., 2019) and backpropagate the loss only through the last b iterations ($b \ll n$).

Notice that the KVs used in the last iteration come from the second-to-last iteration, so if we set $b = 1$, then backpropagation would not reach the

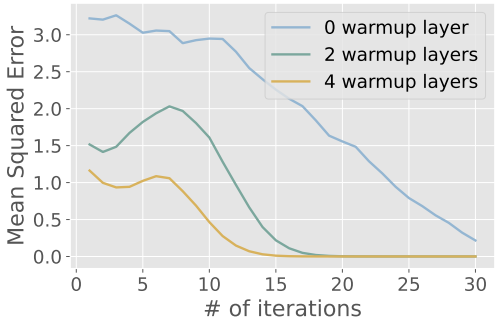


Figure 3: MSE of the KV before and after the ith iteration. The model is randomly initialized and tested with 2048 tokens.

KVs and hence the model parameters used to calculate the KVs are not trained at all, which would result in a large performance degradation. We empirically find that with $b \geq 2$, the performance of our model is comparable with that of a standard transformer. To reduce memory consumption, we set $b = 2$ by default, which means we only back-propagate through two iterations.

2.2.3 Forward Propagation: Fast Convergence of KV

With gradient stopping, the first $n - b$ iterations are solely used in forward propagation to compute KVs that are fed into the last b iterations. When n is large, it is still too costly to run $n-b$ iterations of forward propagation. Fortunately, we observe that the values of KVs converge very fast over iterations and hence we do not have to run $n - b$ iterations to obtain the final KVs. Figure 3 shows the convergence of KVs of a randomly initialized model with the same configuration as TinyLlama (Zhang et al., 2024). The input is a randomly picked segment of text from the MiniPile (Kaddour, 2023) dataset with 2048 tokens (i.e., $n = 2048$). We measure the change of KVs over consecutive iterations using the mean squared error. As can be seen, KVs converge after only a few tens of iterations, with more warmup layers leading to faster convergence. Therefore, we use m iterations ($m \ll n$) to approximate the KVs of $n - b$ iterations. We empirically find that $m = 7$ is sufficient for model training and larger values of m do not further improve the performance.

2.3 Inference with Prompts

It is straightforward to employ our model for autoregressive generation. However, our model cannot do parallel encoding of prompts like standard transformers for the same reason as it cannot do parallel training. Fortunately, since iterative computation of KVs is fast to converge, we can just perform iterative computation for the prompts for $m + b$ iterations. Typically, $m + b$ is far less than the number of tokens to generate, and thus the extra time spent in encoding the prompts is negligible.

3 Experiments

We empirically verify the effectiveness of our method on the Llama model (Touvron et al., 2023). We show that our method achieves significant memory reduction and throughput improvement as well as competitive performance in language modeling and downstream tasks compared with standard transformers. We also show that our method could effectively integrate with other memory-saving techniques.

3.1 Generation Throughput

We test our method with 1.1B, 7B, and 30B parameters on an NVIDIA GeForce RTX 3090 (24GB) GPU and an NVIDIA A100 (80GB) GPU respectively. The 1.1B model configuration follows that of TinyLlama (Zhang et al., 2024) and the 7B and 30B model configuration follows that of the original Llama (Touvron et al., 2023). We set $m = 7, b = 2$ and $w = \{2, 10\}$. Our implementation is based on HuggingFace Transformers (Wolf et al., 2020) with kernel replacement with FlashAttention 2 (Dao, 2023), fused RMS norm, fused cross-entropy and fused SwiGLU.

Following FlexGen (Sheng et al., 2023), the evaluation is conducted in an end-to-end fashion. For a prompt of length s, we let the model generate output sequences of length n with batch size b. The latency t is defined as the total time in seconds spent in processing the prompts and generating all the bn tokens. The generation throughput is defined as bn/t tokens per second.

Table 1 compares the maximum batch sizes and throughput of standard Llama models and our models on the two types of GPUs. Note that some of the sequence lengths exceed the maximum input lengths that the models are trained on, but that does not affect batch size and throughput measurement. We still benchmark the batch sizes and throughput to show the potential of our method on other models allowing larger sequence lengths. It can be seen that our method achieves significantly larger batch sizes and higher throughput on all of the settings.

GPU	Model Size	Seq. Length	Batch Size			Throughput (tokens/s)		
			Llama	Ours $w = 2$	Ours $w = 10$	Llama	Ours $w = 2$	Ours $w = 10$
RTX 3090	1.1B	5+8187	48	384 (8×)	119 (2.5×)	1424.96	4113.37 (2.9×)	2374.05 (1.7×)
		5+2043	239	1150 (4.8×)	289 (1.2×)	5142.86	10033.40 (2.0×)	7239.92 (1.4×)
	7B	5+8187	1	12 (12×)	4 (4×)	32.02	151.91 (4.7×)	83.80 (2.6×)
		2048+2048	2	23 (11.5×)	8 (4×)	56.98	171.65 (3.0×)	119.68 (2.1×)
		5+2043	5	64 (12.8×)	16 (3.2×)	140.88	534.02 (3.8×)	315.38 (2.2×)
		512+512	9	95 (10.6×)	32 (3.6×)	225.31	378.89 (1.7×)	380.60 (1.7×)
		512+1024	7	72 (10.3×)	16 (2.3×)	174.11	401.92 (2.3×)	310.05 (1.8×)
	30B (CPU-offload)	512+1024	4	83 (20.8×)	23 (5.8×)	0.23	5.99 (26.0×)	1.63 (7.1×)
A100	7B	2048+2048	15	128 (8.5×)	42 (2.8×)	141.10	421.02 (3.0×)	315.09 (2.2×)
	30B	2048+2048	1	32 (32×)	8 (8×)	14.10	108.29 (7.7×)	77.65 (5.5×)

Table 1: Maximum generation batch size and throughput on an RTX 3090 (24GB) and an A100 (80GB) GPU respectively with different sequence lengths. Following Zhang et al. (2023), we use "$x + y$" to denote a prompt length of x and a generation length of y.

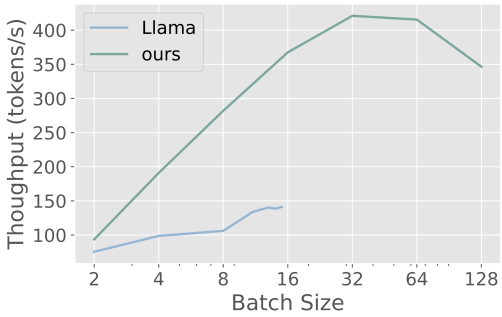


Figure 4: Throughput of 7B Llama and our model w.r.t. the batch size.

Notice that the maximum throughput is not necessarily achieved using the maximum batch size. Figure 4 shows the throughput of 7B Llama and our model on an A100 GPU w.r.t. the batch size. The prompt length and generation length are both 2048. We find that when the batch size grows larger than 32, the throughput no longer increases and even decreases with a batch size of 128. This may indicate that the model operations are turning from memory-bound into compute-bound (Dao et al., 2022) and the throughput is limited by the computation power. Further improvement in throughput may require more efficient computation kernels instead of larger batch sizes.

We also would like to mention that the increased throughput is not necessarily due to the increased batch size. As shown in Figure 4, our model has much higher throughput than Llama even with the same batch sizes. We leave the detailed analysis to Appendix C.6.

3.2 Model Performance

To evaluate the performance of our model in language modeling and downstream tasks, we pre-train from scratch two 1.1B models with $m = 7$, $b = 2$ and $w = \{2, 10\}$. We use TinyLlama as our baseline, whose size is also 1.1B. We pre-train the models on a 100B subset of the SlimPajama dataset (Soboleva et al., 2023). The training details are consistent with those of TinyLlama (Zhang et al., 2024). All models are trained with AdamW (Loshchilov and Hutter, 2019) with $\beta_1 = 0.9$ and $\beta_2 = 0.95$. The batch size is 2M tokens. We use a cosine learning rate schedule with a maximum learning rate of 4.0×10^{-4} and a warmup of 200 steps. The final learning rate is 4.0×10^{-5}. We use a weight decay of 0.1 and gradient clipping of 1.0. The models are trained on 128 NVIDIA A800 (80GB) GPUs.

During evaluation, we perform inference in the standard left-to-right fashion instead of using the method of Section 2.3. We report the perplexity on a 10M subset of the development set of SlimPajama. We also test the zero-shot performance on commonsense reasoning tasks following Zhang et al. (2024), including Hellaswag (Zellers et al., 2019), OpenBookQA (Mihaylov et al., 2018), WinoGrande (Sakaguchi et al., 2021), ARC-Easy and ARC-Challenge (Clark et al., 2018), BoolQ (Clark et al., 2019a), and PIQA (Bisk et al., 2020). The tests are conducted using the lm-eval-harness framework (Gao et al., 2023). For these tasks, we encode the prompts with the same number of iterations ($m + b = 9$) as in training.

Model	HellaSwag	Obqa	WinoGrande	ARC-c	ARC-e	BoolQ	PIQA	Avg
TinyLlama	44.58	30.2	50.99	25.00	46.38	60.46	68.93	46.65
Ours ($w = 2$)	42.22	30.6	49.64	24.74	43.10	61.38	66.49	45.45
Ours ($w = 10$)	44.74	31.0	51.70	24.83	46.38	61.38	67.90	46.84

Table 2: Zero-shot accuracy on commonsense reasoning tasks.

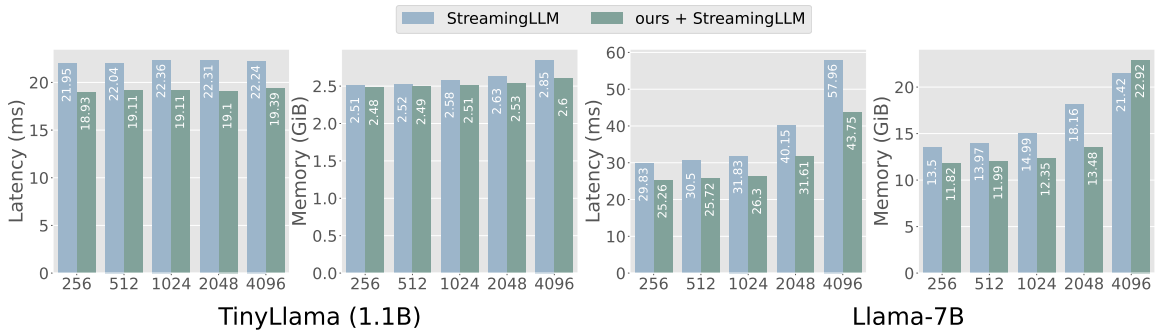


Figure 5: Comparison of latency per token and memory consumption of StreamingLLM and our model ($w = 10$) integrated with StreamingLLM w.r.t. different cache sizes.

Model	Dev ppl.
TinyLlama	9.219
Ours ($w = 2$)	9.746
Ours ($w = 10$)	9.265

Table 3: Perplexity on a 10M subset of the development set of SlimPajama.

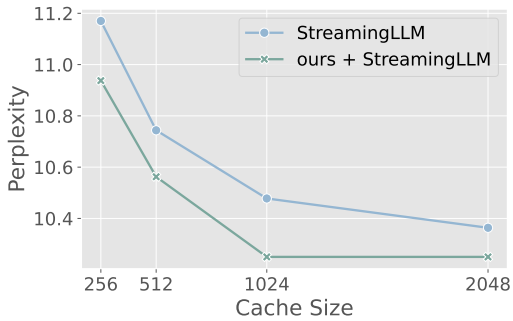


Figure 6: Comparison of StreamingLLM and our model integrated with StreamingLLM w.r.t. the cache size. We use 4 initial tokens for all settings. The results are collected on the first text sample of PG19 (Rae et al., 2020).

Table 2 and 3 show the results. The performance of our models is comparable to that of TinyLlama. In particular, our model with $w = 10$ has almost no performance degradation, while achieving significantly higher generation throughput as evaluated in Section 3.1. Our model with $w = 2$ has a small but noticeable decrease in performance for most of the tasks, but it achieves even higher increase in throughput.

Despite the competitive performance and higher inference efficiency of our models, we note that pre-training our model costs about 3 times the time of pre-training TinyLlama with the same amount of data due to the iterative training process. Nevertheless, we believe that in most scenarios, a speedup in inference is worth a slowdown in training which is a one-time process.

3.3 Integration with StreamingLLM

We have previously mentioned that our method is orthogonal to other memory-saving techniques and can be easily integrated with them. Here we integrate our method with StreamingLLM (Xiao et al., 2024). StreamingLLM employs an attention sink that only preserves the KV cache of the first few tokens (four by default) and recent tokens, which empowers LLMs to process infinite-length inputs.

As shown in Figure 5, the integration of StreamingLLM and our model ($w = 10$) achieves lower latency and memory consumption compared to the original StreamingLLM on different cache sizes (numbers of cached tokens).

We further showcase that integration with our method does not hinder the ability of StreamingLLM to process infinite-length tokens. Specifically, we integrate StreamingLLM into our model ($w = 10$) trained in Section 3.2

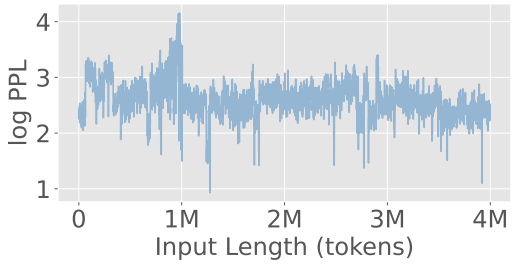


Figure 7: Language modeling perplexity of our model integrated with StreamingLLM on texts with 4M tokens. Following Xiao et al. (2024), we use the concatenated test set of the PG19 dataset (Rae et al., 2020) as the input.

Model Size	Dev ppl.		
	all-bottom	all-top	sandwich
50M	14.556	221.850	14.069
1.1B	7.668	9.098	7.381

Table 4: Model performance with different warmup layer placements ($w = 2$).

with different cache sizes and find that the integrated model achieves even lower perplexities than StreamingLLM as shown in Figure 6. We also let the model handle inputs with a sequence length of four million tokens. As shown in Figure 7, the integrated model can effectively process the input with the perplexity remaining stable.

4 Analyses

In this section, we empirically analyze design choices in our method. For experiment details, please refer to Appendix B.

4.1 The Sandwich Configuration

In Section 2.1, we propose to add some warmup layers to improve model performance. Note that inference efficiency is determined by the number of warmup layers w and not by their placement. Here we ask the following question: *with the number of warmup layers fixed, where should we place the warmup layers to achieve the best performance?*

Table 4 shows the model performance in language modeling with two warmup layers (i.e., $w = 2$) that are placed at the bottom, at the top, and with the default sandwich configuration. It can be seen that the sandwich style performs the best. A possible explanation is that top and bottom layers serve different functionalities (e.g., semantic vs. syntactic) and it makes more sense to warm up both than just one of them.

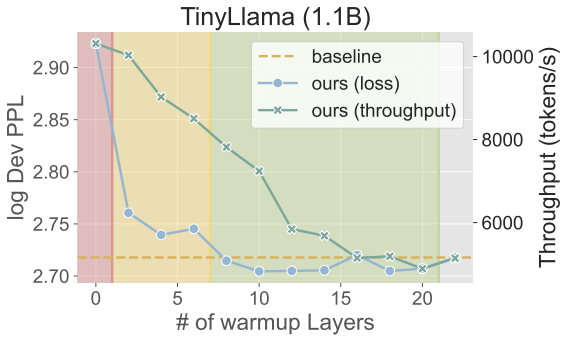


Figure 8: Effect of the number of warmup layers on model performance and throughput. The right-most point ($w = 22$) denotes that all the layers are warmup layers and hence our model becomes exactly the standard transformer (the baseline). The throughput is tested on an RTX 3090 GPU with prompt length 5 and generation length 2043.

4.2 Number of Warmup Layers

Warmup layers serve as a bridge between the standard transformer and our model. The more warmup layers we keep, the more similar it is to the standard transformer, and the less memory we save. In this section, we ask the following question: *how does the number of warmup layers w affect the model performance and throughput?*

We test the model with 1.1B (22 layers) parameters and different numbers of warmup layers (Figure 8). Surprisingly, we find that the log dev perplexity does not monotonically decrease with the number of warmup layers. Without warmup layers (the red region), the model performance is significantly worse than that of the standard transformer. With only a few warmup layers (the yellow region), the model performance is greatly improved and close to that of the standard transformer, but its throughput is decreased at the same time. With more warmup layers (the green region), the model even outperforms the standard transformer, while suffering from further decrease in throughput.

The results point to a trade-off between model performance and throughput that is controllable by the number of warmup layers. If pursuing a high throughput, one could keep only a few warmup layers (the yellow region). If good performance is crucial, one could keep more warmup layers (the left part of the green region).

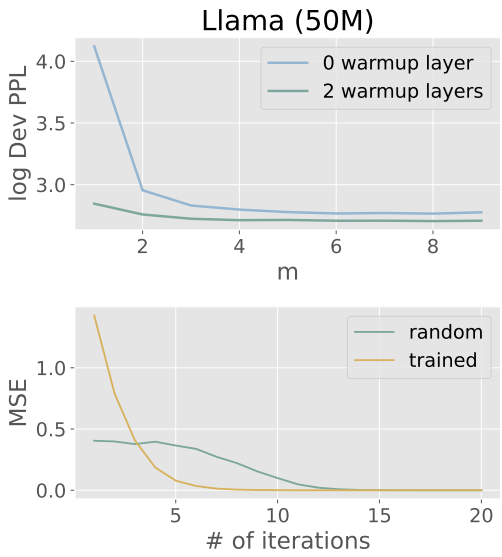


Figure 9: (Top) Experiment on a Llama (50M) with different values of m during training. (Bottom) MSE of the KV before and after the ith iteration. The models are tested with 1024 tokens.

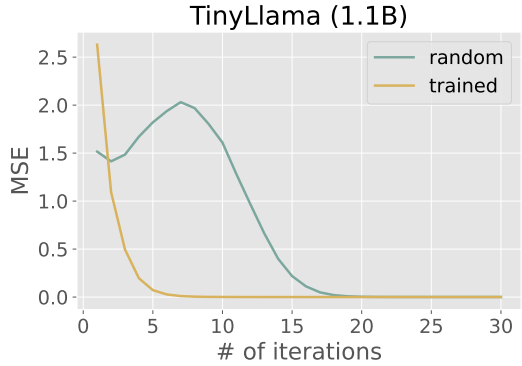


Figure 10: MSE of the KV before and after the ith iteration. The models are tested with 2048 tokens.

4.3 Convergence of KV

In Section 2.2.3, we have shown that KVs converge very fast for a random model and hence we use $m \ll n$ iterations to compute KVs. Here, we ask the following questions: *how fast do KVs converge for a trained model and what value shall we pick for m?*

We measure the convergence of KVs of a 50M Llama (Figure 9, bottom) and the 1.1B model ($w = 2$) pre-trained in Section 3.2 (Figure 10). It can be seen that while a random model requires 15–20 iterations to converge, a trained model requires far fewer iterations. This hints at a small value of m especially during late stages of training. We then evaluate model performance when trained with different values of m (Figure 9, top). It can be seen that the model performance converges with $m \geq 6$, with more warmup layers leading to faster convergence. This justifies our default choice of $m = 7$.

5 Related Work

Extensive research has been done on reducing KV cache for efficient inference of LLMs. With the exception of vLLM (Kwon et al., 2023), which proposes paged attention to reduce memory fragmentation of the KV cache from a system perspective, most recent works focus on compressing the KV cache by reducing the length of the cached KV sequence. Jiang et al. (2023a,b) accelerate model inference by compressing document-level prompts into short prompts. Li et al. (2023) remove the redundancy in the input context. Mu et al. (2023) train gist tokens to replace the reusable system prompts. Ren et al. (2023) incrementally compress a specified span of tokens into compact ones to reduce the KV cache length. Liu et al. (2023) find that only pivotal tokens have a significant influence at a future step, so pruning unimportant tokens does not affect the performance. Ge et al. (2023) argue that the attention heads could be classified into different types and propose to apply different KV pruning strategies to different types of attention heads. Xiao et al. (2024); Han et al. (2023) find that only initial tokens and recent tokens are crucial and propose to store only the KVs of these tokens to enable infinite-length context for LLMs. Zhang et al. (2023) propose a KV cache eviction policy based on the summation of attention scores to only keep a small portion of the KV cache in memory. Unlike these previous methods, our method reduces the memory consumption of the KV cache by reducing the number of layers, which is orthogonal to these methods and can potentially be combined with these methods to further reduce the KV cache and improve inference efficiency.

Feedback Transformers (Fan et al., 2020) aggregate hidden representations from all layers and use them as token memory. This is followed by KV projections to obtain the key-value pairs for all layers. Their experimental results show improved performance with this strategy, even when using only the hidden representation from the top layer. However, their sequential training process is time-costly and not practical for large models. Our method supports parallel training, which is more efficient and scalable.

6 Conclusion

In this paper, we propose a novel method to reduce the memory consumption and improve the throughput of LLMs by reducing the number of layers whose keys and values need to be computed and cached. We empirically show that our method achieves significant memory reduction and throughput improvement with negligible performance degradation. We also show that our method could effectively integrate with other memory-saving techniques like StreamingLLM. Future work includes designing more efficient training approaches, developing large-batch-friendly kernels, and verifying our method on larger and more complex LLMs. We hope that our work could provide a new perspective for improving inference efficiency of LLMs and inspire more research in this direction.

Limitations

Though our method achieves impressive memory reduction and throughput improvement, we would like to point out its limitations from the following aspects:

- Due to the iterative training, our model requires about $3\times$ the time to pre-train a model with the same amount of data. In other words, our method improves the inference efficiency at the cost of the training efficiency. A potential remedy is that if one has a pre-trained model, one could use it to initialize our model, which is empirically found to speed up the process of training.

- Since our method requires iteratively processing the prompts, the throughput degrades when the prompts are much longer than the generation length, e.g., in document summarization. Generally, our method is more suitable for tasks with a large generation length, such as translation, dialogue, question answering, CoT problem solving, etc.

References

Muhammad Adnan, Akhil Arunkumar, Gaurav Jain, Prashant J Nair, Ilya Soloveychik, and Purushotham Kamath. 2024. Keyformer: Kv cache reduction through key tokens selection for efficient generative inference. *arXiv preprint arXiv:2403.09054*.

Yonatan Bisk, Rowan Zellers, Jianfeng Gao, Yejin Choi, et al. 2020. Piqa: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pages 7432–7439.

Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. 2019a. BoolQ: Exploring the surprising difficulty of natural yes/no questions. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 2924–2936, Minneapolis, Minnesota. Association for Computational Linguistics.

Kevin Clark, Urvashi Khandelwal, Omer Levy, and Christopher D. Manning. 2019b. What does BERT look at? an analysis of BERT's attention. In *Proceedings of the 2019 ACL Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*, pages 276–286, Florence, Italy. Association for Computational Linguistics.

Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457*.

Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc Le, and Ruslan Salakhutdinov. 2019. Transformer-XL: Attentive language models beyond a fixed-length context. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 2978–2988, Florence, Italy. Association for Computational Linguistics.

Tri Dao. 2023. Flashattention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691*.

Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*.

Angela Fan, Thibaut Lavril, Edouard Grave, Armand Joulin, and Sainbayar Sukhbaatar. 2020. Addressing some limitations of transformers with feedback memory. *arXiv preprint arXiv:2002.09402*.

Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac'h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. 2023. A framework for few-shot language model evaluation.

Suyu Ge, Yunan Zhang, Liyuan Liu, Minjia Zhang, Jiawei Han, and Jianfeng Gao. 2023. Model tells you what to discard: Adaptive KV cache compression for

LLMs. In *Workshop on Advancing Neural Network Training: Computational Efficiency, Scalability, and Resource Optimization (WANT@NeurIPS 2023)*.

Chi Han, Qifan Wang, Wenhan Xiong, Yu Chen, Heng Ji, and Sinong Wang. 2023. Lm-infinite: Simple on-the-fly length generalization for large language models. *arXiv preprint arXiv:2308.16137*.

Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2023a. LLMLingua: Compressing prompts for accelerated inference of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 13358–13376, Singapore. Association for Computational Linguistics.

Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2023b. Longllmlingua: Accelerating and enhancing llms in long context scenarios via prompt compression. *arXiv preprint arXiv:2310.06839*.

Jean Kaddour. 2023. The minipile challenge for data-efficient language models. *arXiv preprint arXiv:2304.08442*.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626.

Yucheng Li, Bo Dong, Frank Guerin, and Chenghua Lin. 2023. Compressing context to enhance inference efficiency of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 6342–6353, Singapore. Association for Computational Linguistics.

Zichang Liu, Aditya Desai, Fangshuo Liao, Weitao Wang, Victor Xie, Zhaozhuo Xu, Anastasios Kyrillidis, and Anshumali Shrivastava. 2023. Scissorhands: Exploiting the persistence of importance hypothesis for LLM KV cache compression at test time. In *Thirty-seventh Conference on Neural Information Processing Systems*.

Ilya Loshchilov and Frank Hutter. 2019. Decoupled weight decay regularization. In *International Conference on Learning Representations*.

Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2017. Pointer sentinel mixture models. In *International Conference on Learning Representations*.

Todor Mihaylov, Peter Clark, Tushar Khot, and Ashish Sabharwal. 2018. Can a suit of armor conduct electricity? a new dataset for open book question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2381–2391, Brussels, Belgium. Association for Computational Linguistics.

Jesse Mu, Xiang Lisa Li, and Noah Goodman. 2023. Learning to compress prompts with gist tokens. In *Thirty-seventh Conference on Neural Information Processing Systems*.

Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. 2023. Efficiently scaling transformer inference. *Proceedings of Machine Learning and Systems*, 5.

Jack W. Rae, Anna Potapenko, Siddhant M. Jayakumar, Chloe Hillier, and Timothy P. Lillicrap. 2020. Compressive transformers for long-range sequence modelling. In *International Conference on Learning Representations*.

Machel Reid, Edison Marrese-Taylor, and Yutaka Matsuo. 2021. Subformer: Exploring weight sharing for parameter efficiency in generative transformers. In *Findings of the Association for Computational Linguistics: EMNLP 2021*, pages 4081–4090, Punta Cana, Dominican Republic. Association for Computational Linguistics.

Siyu Ren, Qi Jia, and Kenny Zhu. 2023. Context compression for auto-regressive transformers with sentinel tokens. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 12860–12867, Singapore. Association for Computational Linguistics.

Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. 2021. Winogrande: An adversarial winograd schema challenge at scale. *Communications of the ACM*, 64(9):99–106.

Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Re, Ion Stoica, and Ce Zhang. 2023. FlexGen: High-throughput generative inference of large language models with a single GPU. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 31094–31116. PMLR.

Daria Soboleva, Faisal Al-Khateeb, Robert Myers, Jacob R Steeves, Joel Hestness, and Nolan Dey. 2023. SlimPajama: A 627B token cleaned and deduplicated version of RedPajama.

Philippe Tillet, Hsiang-Tsung Kung, and David Cox. 2019. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, pages 10–19.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*.

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander Rush. 2020. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, Online. Association for Computational Linguistics.

Haoyi Wu and Kewei Tu. 2023. Probabilistic transformer: A probabilistic dependency model for contextual word representation. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 7613–7636, Toronto, Canada. Association for Computational Linguistics.

Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2024. Efficient streaming language models with attention sinks. In *The Twelfth International Conference on Learning Representations*.

Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. 2019. HellaSwag: Can a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4791–4800, Florence, Italy. Association for Computational Linguistics.

Peiyuan Zhang, Guangtao Zeng, Tianduo Wang, and Wei Lu. 2024. Tinyllama: An open-source small language model. *arXiv preprint arXiv:2401.02385*.

Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Re, Clark Barrett, Zhangyang Wang, and Beidi Chen. 2023. H2o: Heavy-hitter oracle for efficient generative inference of large language models. In *Thirty-seventh Conference on Neural Information Processing Systems*.

A Proof of The Training Theorem

Here we formally prove Theorem 1 in Section 2.2. Remember that we need to compute the loss for each token sequentially, but we propose to train all the tokens in parallel with n iterations.

Theorem 1. *The two computation graphs are equivalent in terms of model training.*

To prove the theorem, we first introduce the following lemma.

Lemma 1. *In the first computation graph, denote the final hidden representation of the i-th token as $h_i = f_{1,i}(x_1, x_2, \ldots, x_i)$. In the second computation graph, denote the final hidden representation of the i-th token in iteration t as $h_i^{(t)} = f_{2,i}^{(t)}(x_1, x_2, \ldots, x_i)$. We have $f_{1,i} = f_{2,i}^{(t)}, \forall t \geq i$, no matter what the initial KVs are.*

Proof. In the first computation graph, denote the KVs of the i-th token as KV_i and that under parallel training with iteration t as $KV_i^{(t)}$.

Since the basic networks are the same for the two computation graphs, we further denote g_i as $h_i = f_{1,i}(x_1, x_2, \ldots, x_i) = g_i(x_1, x_2, \ldots, x_i, KV_1, KV_2, \ldots, KV_{i-1})$, as well as $h_i^{(t)} = f_{2,i}^{(t)}(x_1, x_2, \ldots, x_i) = g_i(x_1, x_2, \ldots, x_i, KV_1^{(t-1)}, \ldots, KV_{i-1}^{(t-1)})$. The only difference in the computation of the final hidden representations lies in the KVs.

We prove the lemma by induction. For the base case, since we have removed the diagonal of the attention matrix, h_1 and $h_1^{(t)}$ does not rely on any key or value. So we have $h_1 = f_{1,1}(x_1) = g_i(x_1) = f_{2,1}^{(t)}(x_1) = h_1^{(t)}, \forall t$. That is $f_{1,1} = f_{2,1}^{(t)}, \forall t$. Since the computation graphs of the first token are the same, KV_1 and $KV_1^{(t)}$ are the same for all t.

For the inductive step, we assume that $\forall i \leq T, f_{1,i} = f_{2,i}^{(T)} = g_i, KV_i = KV_i^{(T)}$. Then for iteration $T+1$:

$\forall i \leq T+1$, the computation of the i-th token relies on the KVs of the first $i-1$ tokens. That is, $h_i^{(T+1)} = f_{2,i}^{(T+1)}(x_1, x_2, \ldots, x_i) = g_i(x_1, x_2, \ldots, x_i, KV_1^{(T)}, \ldots, KV_{i-1}^{(T)})$.

Since $i \leq T+1$, we have $i-1 \leq T$. By induction we have $KV_{i-1}^{(T)} = KV_{i-1}$. Thus, we have $g_i(x_1, x_2, \ldots, x_i, KV_1^{(T)}, \ldots, KV_{i-1}^{(T)}) = g_i(x_1, x_2, \ldots, x_i, KV_1, \ldots, KV_{i-1}) = f_{1,i}(x_1, x_2, \ldots, x_i) = h_i$. The computation graphs are the same for the first $T+1$ tokens, then we have $KV_i = KV_i^{(T+1)}$.

Thus, we have $f_{1,i} = f_{2,i}^{(t)}, \forall i \leq t$. The proof does not rely on initial KVs, so the conclusion holds for any initial KVs. □

Let $t = n$, we have the entire computation graphs are equivalent. Therefore, we have proved Theorem 1.

From the lemma, we could learn one more thing: The parallel training has the same computation graph as the sequential training when the iteration $t \geq n$. This indicates that the parallel training is theoretically guaranteed to converge to the same solution as the sequential training. Once it is converged, it will not diverge. Our work finds that the KVs converge much faster than the theoretical n

Model Size	50M	1.1B	7B	30B
Hidden Size	512	2048	4096	6656
Intermediate Size	1024	5632	11008	17920
Max Trained Length	1024	2048	–	–
# Layers	8	22	32	60
# Attention Heads	8	32	32	52
# KV Heads	4	4	32	52
RMS Norm eps	1e-5	1e-5	1e-6	1e-6
Vocab Size		32000		

Table 5: Model configurations.

Section	4.1		4.2	4.3
Model Size	50M	1.1B	1.1B	50M
m	7	7	7	–
b	2	2	2	2
w	2	2	–	2
lr scheduler		cosine		
max. lr	3e-4	3e-4	4e-4	3e-4
min. lr	0	0	4e-5	0
optimizer		AdamW		
β_1	0.9	0.9	0.9	0.9
β_2	0.999	0.999	0.95	0.999
batch size (tokens)	16K	256K	256K	16K
warmup ratio	0.015	0.015	0.024	0.015
weight decay	6.6e-6	6.6e-6	1e-1	6.6e-6
gradient clipping	1.0	1.0	1.0	1.0
initialize from pre-trained	yes	yes	no	no
epochs	3	1	2B tokens	3
Data	WikiText-103	MiniPile	SlimPajama	WikiText-103
GPU	RTX 3090x1	A100x8	A800x16	RTX 3090x1

Table 6: Training details for Section 4.

iterations, which significantly reduces the training time.

B Model and Training Details

We provide the model configurations and training details in Table 5 and 6. The 7B and 30B model configurations are consistent with those of the original Llama (Touvron et al., 2023). The 1.1B model configuration follows that of TinyLlama (Zhang et al., 2024). We use the WikiText-103 (Merity et al., 2017) (licensed under CC-BY-SA 3.0), MiniPile (Kaddour, 2023) (licensed under MIT) and SlimPajama (Soboleva et al., 2023) (various licenses depending on the data source) as our datasets. Our use of the datasets is consistent with their intended use.

The training of TinyLlama in Section 3 takes 14:42:59, while our models take 1 day, 16:44:16 (2.77x, $w = 2$) and 1 day, 15:52:38 (2.71x, $w = 10$), respectively. The training took place before we optimize our codes, so the training time could be further reduced, especially for the model with $w = 10$.

C More Analyses

In this section, we provide more analyses beyond those in Section 4.

C.1 Initialize with Pre-trained Models

Since our model structure resembles that of the standard transformer, we could initialize our model

Model	Dev ppl.	Avg.
TinyLlama	9.219	46.65
Ours ($w = 2$)	9.746	45.45
TinyLlama-2.5T	7.822	53.97
TinyLlama-500B	9.046	48.28
Ours ($w = 2$, init w/ 2.5T)	8.514	49.55

Table 7: Dev ppl: Perplexity on a 10M subset of the validation set of SlimPajama. Avg: Average zero-shot accuracy on commonsense reasoning tasks. Models are trained on a 100B subset of SlimPajama. The models with italic fonts are from TinyLlama checkpoints.

with pre-trained models. For W_K, W_V of the middle layers, we just ignore the parameters. Experiments show that initialization with pre-trained models could effectively speed up the training process (Table 7). Though our model has a different computation graph from the standard transformer, it could still benefit from the pre-trained models. Our model initialized with a TinyLlama checkpoint trained on 2.5T tokens achieves a perplexity of 8.514, which is much better than the randomly initialized model. It is even better than the TinyLlama checkpoint trained on 500B tokens. Thus, if the pre-trained models are available, initializing our model with them could save a lot of training time.

C.2 Iterations with Gradients

In Section 2.2.2 we set $b = 2$ by default. What if we use larger b? To make sure that the inference process is equivalent (specifically, encoding the prompts), we fix $m + b = 9$ and experiment with different values of b.

Figure 11 shows the perplexity of a 50M Llama with different values of b. Though from $b = 2$ to $b = 3$ the perplexity decreases, for $b = 4$ the perplexity increases. We further confirm that for $b = 4$ the KVs do not converge as fast as for $b = 2$.

Experiments on a 1.1B model (Table 8) show that the perplexity increases with b and the training time also increases. From the training curve, we find that larger b leads to more unstable training, thus the model is harder to converge. Therefore, we set $b = 2$ by default.

C.3 KV Loss for Less Iterations

In Section 4.3 we have shown that the KVs converge very fast for a trained model, yet we still want to make it converge even faster, saving both training and inference time costs. An intuitive idea

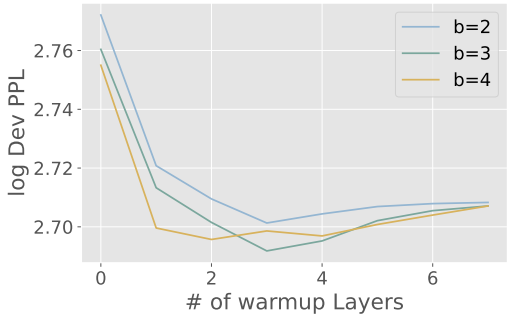


Figure 11: Effect of b on a 50M model with different number of warmup layers.

Model	Dev ppl.	Train Time
Ours ($b = 2$)	10.390	8h
Ours ($b = 3$)	10.476	10h
Ours ($b = 4$)	10.885	13h

Table 8: Effect of b on a 1.1B model.

is to add an MSE loss to the KVs before and after the last iteration to force the KVs to converge. We call this term the "KV Loss". Our experiments (Table 9) show that for small data with small w, the KV loss could lead to better performance. While for large data or large w, the KV loss hurts the performance. This is probably because the KVs are not converged at the beginning of training and the KV loss helps the KVs converge. However, when the KVs are already converged, the KV loss could slow down the training process. In our method, we do not use the KV loss.

C.4 Encode Prompts with Different Number of Iterations

Though we set $m = 7, b = 2$ during training, it does not necessarily mean that we have to encode

Model Size	w	KV Loss	Dev ppl.
50M	0	no	15.965
	0	yes	15.610
	2	no	15.004
	2	yes	15.065
1.1B	2	no	9.746
	2	yes	10.073

Table 9: Effect of the KV loss on a 50M and a 1.1B model. The 50M model is trained on WikiText-103 with 3 epochs and the 1.1B model is trained on a 100B subset of SlimPajama.

Model Size	Model	Batch Size	Latency (s)	Throughput (token/s)
7B	Llama	15	217.71	141.10
	Ours ($w = 2$)	16	89.15	367.56
	Ours ($w = 10$)	16	131.43	249.32
30B	Llama	1	145.23	14.10
	Ours ($w = 2$)	1	101.12	20.25
	Ours ($w = 10$)	1	106.99	19.14

Table 10: Inference latency and throughput of the models with different model sizes. The models are tested on an A100 (80GB) GPU with prompt length 2048 and generation length 2048. The batch size is (approximately) the largest batch size for standard Llama model.

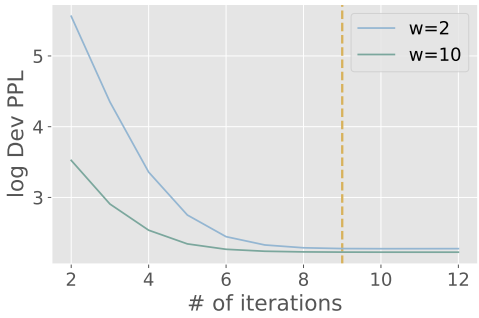


Figure 12: Performance of the models with different numbers of iterations for prompt encoding.

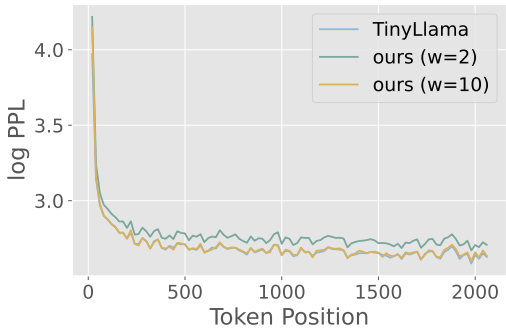


Figure 13: Perplexity of the models with different token positions on PG19. We use the concatenated test set of the PG19 dataset as the input.

the prompts with 9 iterations. Is it possible to encode the prompts with less iterations? What if we encode the prompts with more iterations?

We treat the token segments as prompts and test the model trained in Section 3.2 with different numbers of iterations during encoding. As shown in Figure 12, the perplexity increases when the number of iterations is reduced, but still in a reasonable scale if we only reduce one or two iterations. The more warmup layers there are, the more stable the performance is. Increasing the number of iterations does not noticeably affect the performance. Thus, one could make a trade-off to set the proper number of iterations during inference to balance the time encoding prompts and the quality of generation texts.

C.5 Model Performance With Respect to Token Position

The long-context performance of LLMs highly relies on KV cache (Xiao et al., 2024; Han et al., 2023; Adnan et al., 2024). To verify that the performance of our model does not degrade under long context, we test the perplexity of our model with different token positions on PG19 (Rae et al., 2020). Despite the fact that we only compute the KVs of a few layers, the performance of our model does not degrade with the token position (Figure 13) and is comparable to that of TinyLlama. Due to the limitation of computational resources, we only train models with context length 2048.

C.6 Improvement of Throughput Not Necessarily Due to Larger Batch Size

In Section 3.1, we show that our model achieves higher generation throughput than standard transformers. While one might assume that this improvement is solely due to a larger batch size, Figure 4 demonstrates that our model outperforms the standard transformer even with the same batch size. Additionally, Table 10 reports reduced latency for our model when compared with the standard transformer with the same batch size. This suggests that our method can also benefit scenarios that require fast response. The exact reason for this phenomenon is not yet clear and we speculate that it could be attributed to factors such as the reduced calculation of KVs, the decreased memory consumption enabling faster memory transfer and access, and various implementation details.